

Selective Recurrence Offloading for Online Softmax Merge on RISC-V Vector Clusters

Anonymous Author(s)

Abstract—Blockwise attention repeatedly combines tile results through Online Softmax Merge. Each merge contains two different forms of computation: a state-dependent scalar recurrence over the maximum and normalization state, and a regular $O[D]$ vector update. The recurrence forms a short dependency chain with maximum, exponential, accumulation, and reciprocal operations. The vector phase applies two ready weights independently across D elements. We identify this recurrence–vector computational asymmetry and propose selective recurrence offloading: a Scalar Softmax Merge Unit (SMU) specializes the recurrence, while an existing vector datapath retains the element-wise update. A shared scratchpad transfers the updated state and weights between phases. We instantiate this architecture on the open-source Spatz RISC-V vector cluster without modifying its instruction set or RVV datapath. Across three model-derived merge shapes, the prototype achieves a $4.53\times$ geometric-mean RTL/Verilator cycle-count ratio over a matched-arithmetic RVV baseline. Fitting $C = C_0 + C_s N + C_v N D$ shows a $16.30\times$ reduction in the N -dependent coefficient while preserving the baseline’s vector-scaling behavior. The standalone Scalar SMU maps to 77,103.3 Nangate45 Liberty area units; the evaluated Full-Offload design point requires 48.8% more area. These results show that the specialization boundary inside Online Softmax Merge matters: specializing the recurrence retains an efficient existing vector path and avoids adding standalone vector-update capability.

Index Terms—online Softmax merge, attention, selective offloading, RISC-V vector, scratchpad, accelerator

I. INTRODUCTION

Tiled and blockwise attention reduce data movement by processing the score matrix in tiles and maintaining a compact Online Softmax state [1], [2]. After each tile, a merge updates the running maximum m , normalization term l , and output vector O . This (m, l, O) state lets the computation preserve exact online-normalization structure without materializing the complete score matrix.

Online Softmax Merge is not a uniform computation. Its scalar component updates m and l , rescales the old and tile contributions, and generates two normalization weights. Maximum selection, exponentials, accumulation, and a reciprocal form a state-dependent chain with little data-level parallelism. Its vector component then applies the two weights to $O[D]$. Once the weights are ready, every element in this update is independent and follows the same multiply-add pattern. We call this split the *recurrence–vector computational asymmetry*.

The asymmetry exposes an architectural choice between two familiar execution endpoints. Programmable vector execution preserves flexibility and maps the regular $O[D]$ update onto existing SIMD resources, but it also executes the dependency-bound recurrence through general-purpose instructions. Broad

hardware offload specializes the complete merge, including vector work that an existing vector datapath already handles well. This raises the central question of this paper: must hardware specialize the complete merge, or can it isolate the recurrence while retaining the existing vector engine for regular element-wise work?

We propose a *selective recurrence-offloading architecture*. Its design principle is to specialize the recurrence while reusing the existing vector datapath. A Scalar SMU computes the new scalar state and two weights. It commits these values to a shared scratchpad, after which the vector engine performs the $O[D]$ update. The shared-memory boundary keeps the recurrence engine narrow and avoids a second dedicated vector-update path.

We instantiate this architecture on the open-source Spatz RISC-V vector cluster [3]. The prototype attaches the Scalar SMU as a cluster-local engine with an independent tightly coupled data memory (TCDM) requester. Memory-mapped I/O (MMIO) supplies command and status. Software starts the recurrence phase, observes completion, and then invokes the existing RISC-V Vector (RVV) path. The prototype changes neither instruction decoding nor the RVV datapath.

The measured results connect the architectural partition to its scaling and hardware consequences. Across three representative model-derived (N, D) shapes, Proposed achieves a $4.53\times$ geometric-mean cycle-count ratio over the matched-arithmetic B2R baseline. Proposed reduces fitted C_s by $16.30\times$ and changes fitted C_v by only 6.22% relative to B2R. The Full-Offload design point lowers C_s further, but its C_v is $3.78\times$ Proposed and its standalone mapped area is 48.8% larger. This paper makes three contributions:

- **Architectural observation and specialization boundary.** We identify the recurrence–vector computational asymmetry in Online Softmax Merge and use it to define a selective specialization opportunity.
- **Selective recurrence-offloading architecture.** We combine a recurrence-only Scalar SMU with an existing vector datapath and exchange completed state through a shared scratchpad, avoiding additional dedicated vector-update hardware in the proposed design.
- **Prototype and quantitative evaluation.** We implement the architecture on Spatz without ISA or RVV datapath changes and compare the matched-arithmetic RVV, Proposed, and Full-Offload points using cycle scaling, model-shape, FSM, and standalone-area evidence.

II. BACKGROUND AND MOTIVATION

A. Online Softmax Merge

Blockwise attention produces a tile-local maximum m_{tile} , a normalization term l_{tile} , and an output contribution O_{tile} . The running state contains the corresponding m_{old} , l_{old} , and O_{old} . One merge row computes

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{tile}}), \quad (1)$$

$$s_{\text{old}} = l_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}), \quad (2)$$

$$s_{\text{tile}} = l_{\text{tile}} \exp(m_{\text{tile}} - m_{\text{new}}), \quad (3)$$

$$l_{\text{new}} = s_{\text{old}} + s_{\text{tile}}, \quad (4)$$

$$w_{\text{old}} = s_{\text{old}}/l_{\text{new}}, \quad w_{\text{tile}} = s_{\text{tile}}/l_{\text{new}}, \quad (5)$$

$$O_{\text{new}}[j] = w_{\text{old}}O_{\text{old}}[j] + w_{\text{tile}}O_{\text{tile}}[j], \quad 0 \leq j < D. \quad (6)$$

Equations (1)–(5) produce the updated scalar state and two weights. Equation (6) applies the weights across the output dimension.

B. Recurrence–Vector Computational Asymmetry

The scalar equations form a dependency chain. The maximum determines both exponential arguments. The scaled lengths determine l_{new} , and l_{new} determines the reciprocal used by both weights. The chain combines control-sensitive maximum selection with nonlinear exponential and reciprocal operations. It performs little work per state update, but each step depends on values produced earlier in the same row.

The vector equation has the opposite structure. After weight generation, $O_{\text{new}}[j]$ has no dependence on $O_{\text{new}}[j']$ for $j \neq j'$. Every element executes regular multiply-add arithmetic with unit-stride vector accesses. Its work scales as $O[D]$ and matches a vector load/store and arithmetic pipeline. Figure 1 summarizes this computational split.

C. Execution Alternatives and Specialization Boundary

Programmable execution maps the vector phase efficiently when a processor already provides vector loads, stores, and arithmetic. The same route retains the scalar recurrence in software, where dependency, control, and nonlinear operations limit useful vectorization. The matched-arithmetic B2R baseline in this paper represents this endpoint: software executes the recurrence, and RVV executes the output update.

Full offload places both phases in dedicated hardware. This endpoint can specialize the recurrence and integrate the complete merge behind one command. It also builds standalone vector-update capability alongside an existing vector engine. Our Full-Offload design point represents this alternative. The computational asymmetry suggests an intermediate boundary that dedicates hardware to the recurrence and preserves the vector phase on the existing datapath.

III. SELECTIVE RECURRENCE-OFFLOADING ARCHITECTURE

A. Architecture Overview

Selective recurrence offloading separates the merge at the point where the weights become ready. Figure 2 shows

four platform-independent components: a control plane, a recurrence engine, an existing vector datapath, and a shared scratchpad. The Scalar SMU is the only new compute engine. The architecture retains the vector datapath and its memory path.

B. Execution Partition

The Scalar SMU executes Eqs. (1)–(5). This partition places maximum selection, two exponential rescalings, length accumulation, reciprocal normalization, and weight generation in a short scalar datapath. The engine accepts one or more independent merge rows and produces m_{new} , l_{new} , w_{old} , and w_{tile} for each row.

The existing vector engine executes Eq. (6). It loads the two output vectors and the completed weights, performs the element-wise multiply-add sequence, and stores O_{new} . This assignment keeps the $O[D]$ work on the datapath whose width, memory interfaces, and arithmetic lanes already target regular data parallelism.

C. Shared-State Handoff and Execution Sequence

The architecture uses shared scratchpad state to decouple the two phases. The control plane first makes the old and tile recurrence state visible to the Scalar SMU. The engine reads that state, computes the recurrence, and commits the new state and two weights. Completion establishes that the shared values are ready. The control plane then invokes the vector phase, which consumes the weights and updates $O[D]$.

This handoff keeps the two engines independent at the datapath level. It does not require a private Scalar-SMU-to-vector bypass or a second vector register interface. The shared scratchpad also exposes the state to software and lets the implementation reuse its normal memory arbitration. Platform-specific command registers, requester ports, and memory names belong to the prototype and appear in Section V.

IV. SCALAR SMU MICROARCHITECTURE

A. Scalar Datapath

The Scalar SMU follows the recurrence dependencies directly. Figure 3 maps the implemented RTL dataflow. A comparator selects m_{new} , and two subtractors form the old and tile deltas. Two distinct EXP LUT paths evaluate the rescaling factors in parallel. Two multipliers scale l_{old} and l_{tile} , and an adder forms l_{new} . The implementation registers the scaled lengths and l_{new} before the next FSM state. A single reciprocal path then feeds two weight multipliers to produce w_{old} and w_{tile} .

The RTL receives and writes FP32 values at the scratchpad interface. Internal helpers convert maxima and lengths to signed or unsigned 48-bit fixed point with 32 fractional bits. EXP and reciprocal LUT outputs use Q1.23. The two length-scaling products, l_{new} , and the weights use the 48-bit Q16.32 representation before conversion back to FP32 for writeback. The Scalar SMU contains no $O[D]$ datapath in the proposed scalar-only configuration.

Online Softmax Merge

one merge row

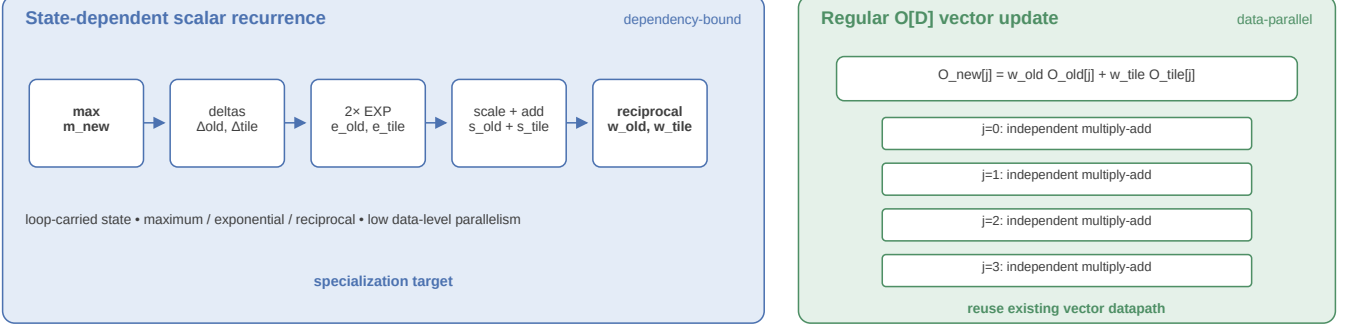


Fig. 1. **Online Softmax Merge contains a dependency-bound recurrence and a data-parallel vector update.** The scalar chain produces two weights; the ready weights expose independent multiply-add work across D elements.

Selective recurrence-offloading architecture

platform-independent

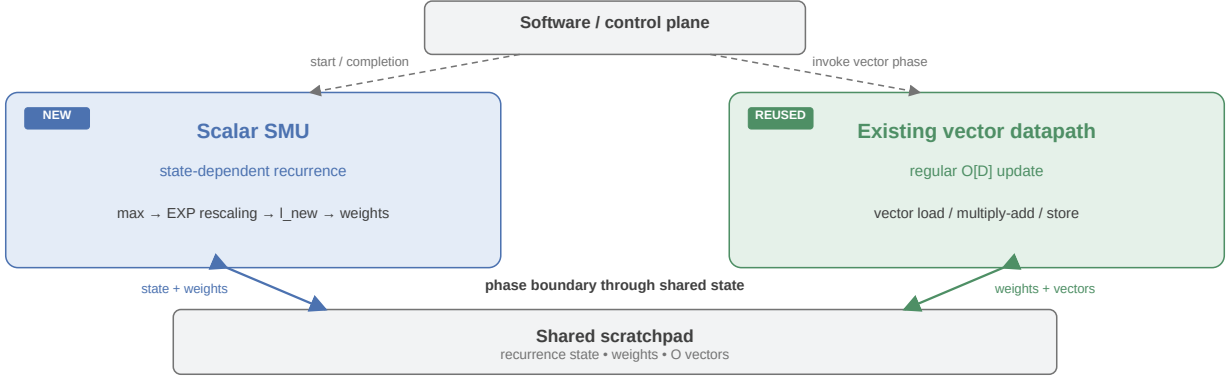


Fig. 2. **Selective recurrence offloading adds a narrow recurrence engine and reuses the existing vector datapath.** Shared scratchpad state forms the phase boundary and removes the need for a separate vector-update path.

B. EXP and Reciprocal Approximation

The EXP block implements a 257-entry LUT over the supported interval $[-8, 0]$ with linear interpolation. Non-positive inputs inside the interval produce a Q1.23 factor; values below -8 saturate to zero, and positive values saturate to one. Online Softmax Merge produces non-positive deltas because m_{new} is the maximum of the two inputs.

The reciprocal block uses a 257-entry mantissa LUT with linear interpolation. It returns a Q1.23 mantissa and a signed exponent adjustment derived from the FP32 exponent of l_{new} . The weight multipliers apply this scaled reciprocal to the two registered lengths. This structure implements the arithmetic used by the RTL and by the matched software reference in the evaluation.

C. Controller and Execution Schedule

The scalar-only controller uses a four-state sequence. `LOAD_SCALAR` issues four scalar scratchpad reads. `COMPUTE_SCALAR` evaluates the maximum, EXP paths, and length update. `COMPUTE_WEIGHT` evaluates the reciprocal and weight generation. `STORE_SCALAR` writes the new maximum, new length, and two weights. Scalar-only execution

then advances to the next row or completion; it skips the RTL's Full-Offload `UPDATE_VECTOR` state.

RTL observer counts give the nominal no-stall schedule in Figure 4: $13 + 1 + 1 + 9 = 24$ busy cycles per row. TCDM handshakes account for 22 cycles, while the two combinational compute stages account for one cycle each. Model-shape observers record 289, 769, and 962 busy cycles for the BERT, Mistral, and Qwen2.5-14B-Instruct shapes, adding one or two wait cycles to $24N$. The schedule describes the SMU busy interval; the measured kernel cycles also include software command and completion handling and the later RVV phase.

V. SPATZ-BASED PROTOTYPE INTEGRATION

A. Relevant Spatz Cluster Organization

Spatz supplies the vector resources that selective recurrence offloading reuses [3]. The evaluated default cluster contains two Snitch-Spatz core complexes and a shared 128-KiB, 16-bank TCDM. Each Spatz vector unit provides a vector arithmetic unit and a vector load/store unit (VLSU) connected to TCDM. The prototype keeps these vector units and their existing TCDM ports unchanged.

Scalar SMU recurrence datapath

RTL structure

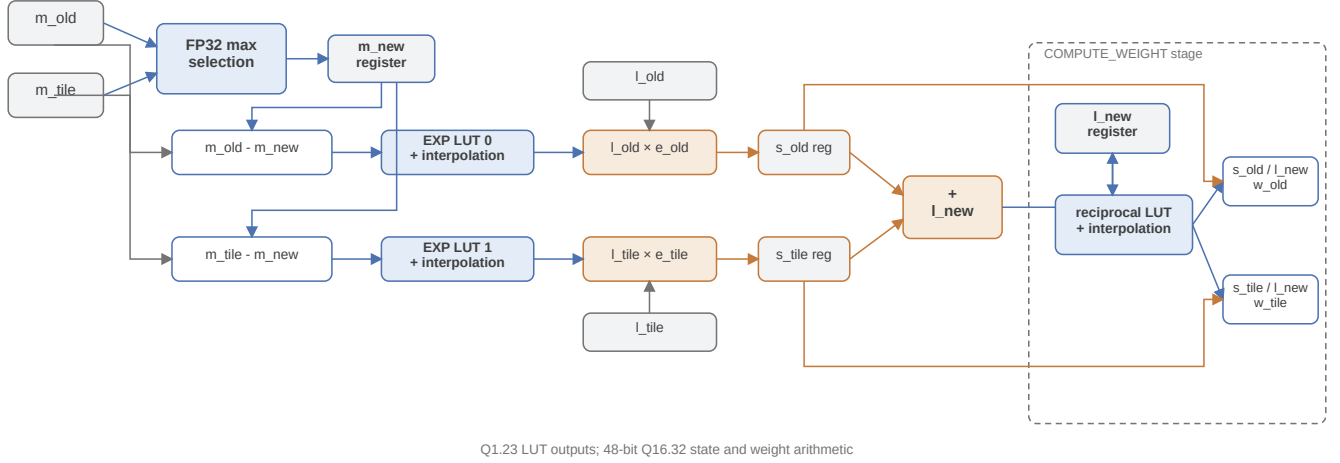


Fig. 3. **The Scalar SMU mirrors the recurrence dependency chain.** Two EXP paths operate in parallel; registered l_{new} then enters one reciprocal path that generates both normalized weights.

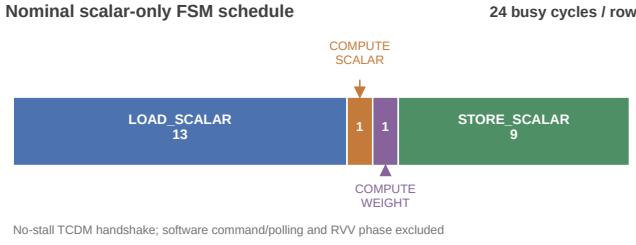


Fig. 4. **The nominal scalar-only FSM requires 24 busy cycles per row:** 13 load, one scalar-compute, one weight-compute, and nine store cycles. The timeline assumes no TCDM stalls.

B. SMU Attachment and TCDM Interface

Figure 5a shows the concrete attachment. The cluster peripheral holds the command, buffer offsets, dimensions, mode, and status fields. It drives the Scalar SMU’s control interface. The SMU adds one independent TCDM requester to the cluster interconnect, alongside the existing core, RVV, and system requesters. This requester performs the scalar reads and writes described in Section IV.

The Scalar SMU reads FP32 arrays for m_{old} , l_{old} , m_{tile} , and l_{tile} . It writes arrays for m_{new} , l_{new} , w_{old} , and w_{tile} . The shared TCDM indexes rows consistently across these arrays. The interconnect arbitrates the requester with the existing memory ports, so the design needs no dedicated physical link from the SMU to Spatz.

C. MMIO Control and Software–Hardware Execution Flow

Software programs TCDM offsets, N , D , stride, and scalar-only mode through MMIO, clears prior completion state, and asserts start. It then polls the busy, done, and error status bits. On completion, the new scalar state and weights already reside in TCDM. Software loads the two weight arrays and calls the

existing RVV kernel for each $O[D]$ row. Figure 5b shows this sequence.

The integration preserves the architectural boundary in a concrete cluster. The Scalar SMU communicates through MMIO and shared TCDM, while the vector phase uses normal RVV instructions and memory accesses. The implementation adds no instruction and changes no RVV arithmetic, load/store, or register path.

VI. EVALUATION

The evaluation asks four architectural questions. Does selective offloading remove the recurrence-related scaling cost? What cycle-count ratios result at representative merge dimensions? What standalone hardware does recurrence specialization require? How does the selected boundary compare with full offload?

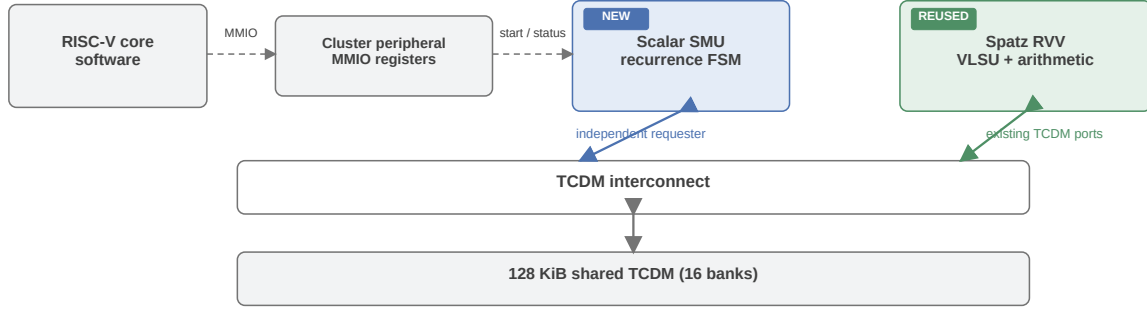
A. Methodology and Baselines

We evaluate FP32 merge kernels with arrays resident in the default Spatz TCDM. The active matrix contains 26 deterministic cases, three configurations, and three independent simulator runs, for 234 records. Each run measures one logical query-position merge in RTL/Verilator. The matched software reference uses the same EXP and reciprocal LUT arithmetic as the RTL. All records pass a scale-aware 10^{-3} functional check, contain finite outputs, and reproduce the same cycle count across the three runs.

The comparison contains three execution points. B2R executes the scalar recurrence in matched-arithmetic software and the vector update on existing RVV. Proposed executes the recurrence in the Scalar SMU and retains the same RVV vector update. Full-Offload executes both phases in the dedicated engine. We report cycle-count ratio as B2R measured cycles divided by design measured cycles, without a frequency conversion.

Spatz prototype integration and execution sequence

a Cluster integration



b Software-hardware sequence

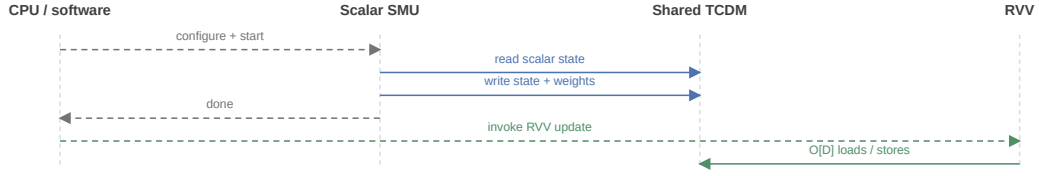


Fig. 5. **Spatz instantiates selective recurrence offloading through its existing peripheral and shared-memory structure.** (a) The Scalar SMU adds an MMIO-controlled TCDM requester; RVV retains its ports. (b) Completion and shared weights sequence the recurrence and vector phases.

The model-derived rows use deterministic generated states and dimensions from BERT, Mistral-7B, and Qwen2.5-14B-Instruct configurations. They represent one attention position across all query heads. N is the query-head count, and D is hidden width per query head. Grouped-query attention therefore uses query heads for N . Standalone hardware results use Yosys/ABC and Nangate45. The area flow reports mapped cells and Liberty cell-area units. A separate pre-layout register-to-register timing flow provides a combinational-delay proxy; the available flow does not define a synchronous $F_{\max}$ or cluster PPA result.

B. Recurrence and Vector Scaling Decomposition

We fit the measured cycles for each design to

$$C(N, D) = C_0 + C_s N + C_v N D \quad (7)$$

using 23 measured (N, D) shapes. C_s captures the N -dependent, recurrence-related scaling component in cycles per row. C_v captures the ND -dependent, vector-related component in cycles per element. C_0 is the fitted intercept and includes fixed effects in this implementation. The three fits have R^2 values between 0.99928 and 0.99999.

Figure 6 shows the fitted coefficients. For B2R, $C_0 = 161.353823$, $C_s = 1471.818713$, and $C_v = 2.267063$. For Proposed, $C_0 = 1382.770246$, $C_s = 90.278155$, and $C_v = 2.126010$. Proposed therefore reduces C_s by $16.30\times$, while its C_v is 6.22% lower than B2R. This combination matches the intended partition: hardware removes most recurrence-related cycle growth, and the reused RVV path continues to determine vector scaling.

For Full-Offload, $C_0 = 1285.577085$, $C_s = 19.317262$, and $C_v = 8.029465$. Its lower C_s shows that broader specialization

reduces the remaining row-dependent term. Its C_v is $3.78\times$ Proposed, placing a larger per-element cost on the design point that replaces the existing RVV update. The scaling result identifies the architectural effect of the boundary: Proposed combines a hardware-like recurrence term with an RVV-like vector term.

C. Representative Model-Derived Shapes

Table I reports the three model-derived dimensions and measured merge-kernel cycles. The dimensions span BERT (12, 64), Mistral (32, 128), and Qwen2.5-14B-Instruct (40, 128). Each configuration processes the same generated state for a given row.

Proposed achieves ratios of $4.81\times$, $4.38\times$, and $4.41\times$ for the three shapes, with a $4.53\times$ geometric mean. Full-Offload achieves a $1.89\times$ geometric mean over the same rows. The larger evaluated $D = 128$ shapes amplify the vector-related coefficient, so retaining the existing RVV path gives Proposed its largest advantage over the Full-Offload point.

D. Hardware Cost

Table II reports standalone block mappings. The Scalar SMU maps to 73,505 cells and 77,103.3 Nangate45 Liberty cell-area units. The Full-Offload block maps to 107,372 cells and 114,713.3 units. Full offload therefore adds 37,610.0 Liberty units relative to the Scalar SMU, an increase of 48.8%.

The Full-Offload block includes dedicated vector-update support as well as a complete control and memory path. Proposed keeps that capability in Spatz and pays only for the recurrence engine. The standalone comparison therefore quantifies the hardware consequence of the specialization boundary at the added-block level.

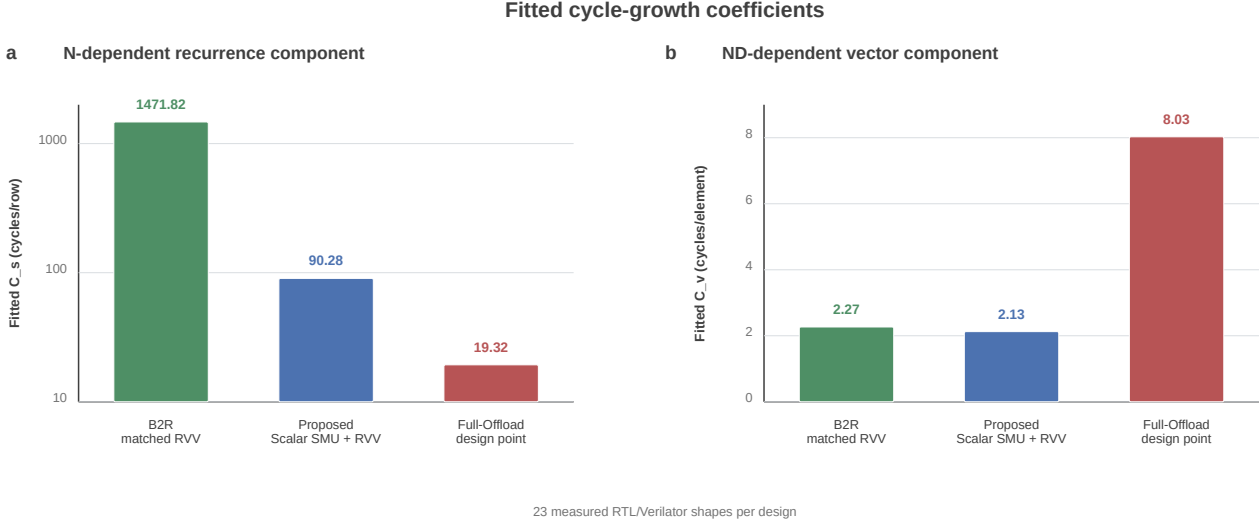


Fig. 6. **Selective recurrence offloading changes the N-dependent scaling component and retains the RVV-like ND-dependent component.** The bars show fitted C_s and C_v from 23 measured RTL/Verilator shapes per design.

TABLE I

MEASURED RTL/VERILATOR MERGE-KERNEL CYCLES FOR DETERMINISTIC STATES AT REPRESENTATIVE MODEL-DERIVED DIMENSIONS. RATIO IS B2R CYCLES DIVIDED BY PROPOSED CYCLES.

Model-derived shape	N	D	B2R	Proposed	Full-Offload	B2R/Proposed
BERT	12	64	19,856	4,128	7,704	$4.81\times$
Mistral	32	128	56,296	12,866	34,728	$4.38\times$
Qwen2.5-14B-Instruct	40	128	69,348	15,733	43,206	$4.41\times$
Geomean	—	—	—	—	—	$4.53\times$

TABLE II

STANDALONE MAPPED HARDWARE. AREA AND THE PRE-LAYOUT DELAY PROXY COME FROM SEPARATE NANGATE45/ABC FLOWS.

Metric	Scalar SMU	Full-Offload
Mapped cells	73,505	107,372
Liberty area units	77,103.3	114,713.3
Relative area	$1.000\times$	$1.488\times$
Reg-to-reg delay proxy	12.34 ns	13.20 ns

E. Specialization Tradeoff

Figure 7 places the three execution points on the same cycle-count-ratio and standalone-added-area view. B2R adds no SMU or accelerator block. Proposed reaches a $4.53\times$ geometric-mean ratio at 77.103 thousand Liberty units. Full-Offload reaches $1.89\times$ at 114.713 thousand units.

More specialization is therefore not the better measured boundary for this kernel. Full-Offload reduces the recurrence coefficient, but it combines a larger vector coefficient with 48.8% more standalone mapped area. Selective recurrence offloading retains the favorable vector scaling of RVV and uses less dedicated hardware in the evaluated design.

VII. RELATED WORK

Programmable and vectorized Softmax implementations keep most computation on the processor side. Titopoulos et al. vectorize FlashAttention with standard RVV instructions

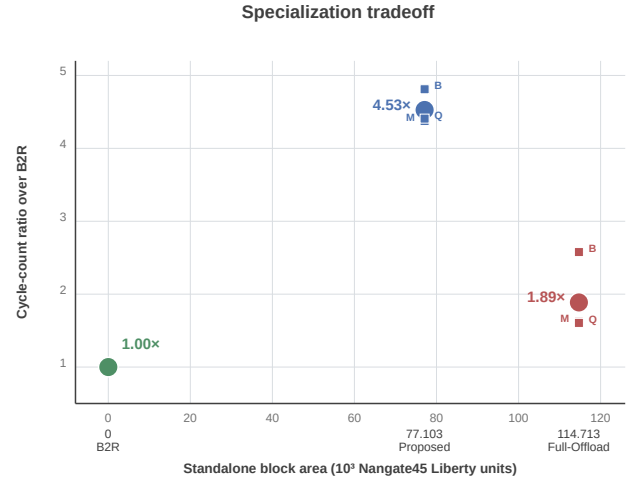


Fig. 7. **Proposed provides the strongest measured cycle-count and standalone-area tradeoff among the evaluated points.** B2R at zero denotes zero incremental accelerator-block area; B, M, and Q mark the three shapes.

[4], and Lai et al. use RVV intrinsics for Softmax [5]. VFA changes the FlashAttention schedule through global-maximum pre-computation to reduce vector work [6]. These approaches preserve programmability and vector reuse. Our architecture retains that vector endpoint while moving the state-dependent merge recurrence to dedicated hardware.

ISA and operator-level acceleration place specialization inside the programmable path. VEXP adds scalar and SIMD exponential operations to RISC-V [7]. Alexandridis et al. fuse exponential and multiplication operators for FlashAttention hardware [8]. These designs accelerate selected arithmetic operations. Selective recurrence offloading instead treats the complete dependency-bound recurrence as the unit of specialization and leaves the vector ISA unchanged.

Dedicated Softmax accelerators choose a broader boundary. Softmax combines unnormalized and normalization hardware [9]. TEA-S implements a compact PLAC-based Softmax architecture [10]. SoftEx attaches a Softmax/GELU hardware processing engine to shared TCDM [11], and Fairoose and Mishra implement parallel online normalization inside a Softmax accelerator [12]. These systems demonstrate several useful points on the dedicated-normalization spectrum. Their specialization scope includes broader Softmax computation than the blockwise recurrence isolated here.

Full-attention accelerators extend specialization across normalization, output production, and surrounding attention operators. Such integration can remove software coordination and exploit operator fusion. It also places the output-vector path within dedicated hardware. The Full-Offload point in our evaluation captures this direction at the merge-kernel level.

Selective recurrence offloading lies between fully programmable vector execution and broad accelerator offload. It specializes the dependency-bound state transition, communicates completed state through a shared scratchpad, and reuses an existing vector datapath for the regular $O[D]$ phase. The Spatz prototype demonstrates this position without defining the architecture around a specific cluster interface.

VIII. CONCLUSION

Online Softmax Merge combines heterogeneous scalar and vector phases. The maximum, exponential rescaling, normalization accumulation, and weight generation form a dependency-bound recurrence. The $O[D]$ update becomes regular data-parallel work once the two weights are ready. Selective recurrence offloading uses this asymmetry to place only the recurrence in a Scalar SMU and retain the existing vector datapath.

The Spatz prototype implements this partition through MMIO control and a shared-TCDM handoff without modifying the ISA or RVV datapath. The measured scaling evidence shows a $16.30\times$ reduction in the recurrence-related C_s term while preserving the RVV-like C_v term. Model-derived shapes reach a $4.53\times$ geometric-mean cycle-count ratio over B2R. Compared with the Full-Offload point, Proposed also avoids the larger vector coefficient and 48.8% additional standalone mapped area. These results establish selective recurrence offloading as a meaningful execution boundary for the evaluated Online Softmax Merge kernel.

REFERENCES

- [1] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.
- [2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 16 344–16 359.

- [3] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025.
- [4] V. Titopoulos, K. Alexandridis, and G. Dimitrakopoulos, "Vectorized FlashAttention with low-cost exponential computation in RISC-V vector processors," *J. Supercomput.*, vol. 82, no. 4, 2026, art. no. 189.
- [5] C.-F. Lai, P.-C. Liu, S.-H. Huang, and H.-H. Chen, "RISC-V vectorized softmax acceleration for IoT edge inference systems," *IEEE Internet of Things Journal*, vol. 13, no. 12, pp. 25 890–25 901, 2026.
- [6] Y. Sun, Y. Li, Z. Zou, B. Du, Z. Zhang, H. Dong, G. Fan, and H. Wang, "VFA: Relieving vector operations in flash attention with global maximum pre-computation," *arXiv:2604.12798*, 2026.
- [7] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "VEXP: A low-cost RISC-V ISA extension for accelerated softmax computation in transformers," in *Proc. IEEE 32nd Symp. Comput. Arithmetic (ARITH)*, 2025, pp. 37–44.
- [8] K. Alexandridis, V. Titopoulos, and G. Dimitrakopoulos, "Low-cost FlashAttention with fused exponential and multiplication hardware operators," in *Proc. IEEE Comput. Soc. Annu. Symp. VLSI (ISVLSI)*, 2025, pp. 1–6.
- [9] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softmax: Hardware/software co-design of an efficient softmax for transformers," in *Proc. 58th ACM/IEEE Design Autom. Conf. (DAC)*, 2021, pp. 469–474.
- [10] Z. Mei, H. Dong, Y. Wang, and H. Pan, "TEA-S: A tiny and efficient architecture for PLAC-based softmax in transformers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3594–3598, 2023.
- [11] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative AI with high-accuracy accelerated softmax and GELU," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025.
- [12] S. Fairoose P. and A. Mishra, "A custom hardware accelerator for softmax function with parallel online normalization," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, pp. 1–13, 2026, early access.